

RELAY — Solution Architecture

PSA Code Sprint 2.0 · Agentic AI in Action

This document covers how RELAY is put together: the execution flow, the main design choices we made, what impact we think it can have, and how we handle security, safety, and scaling.

1. The problem we picked

Most containers at PSA are not simple import/export cargo. They are transshipment — one vessel in, another vessel out, usually on a tight window.

When something breaks — a feeder arrives late, a crane goes down, yard or AIS data goes stale — planners have to work out quickly which connections are actually at risk and what recovery is still physically possible. That information sits across different systems, and the decisions have real cost: missed connections mean extra dwell, rehandling, and downstream schedule knock-on.

We built RELAY around one objective: recover as many connections as we can, without the system quietly dispatching yard moves or cutoff changes on its own.

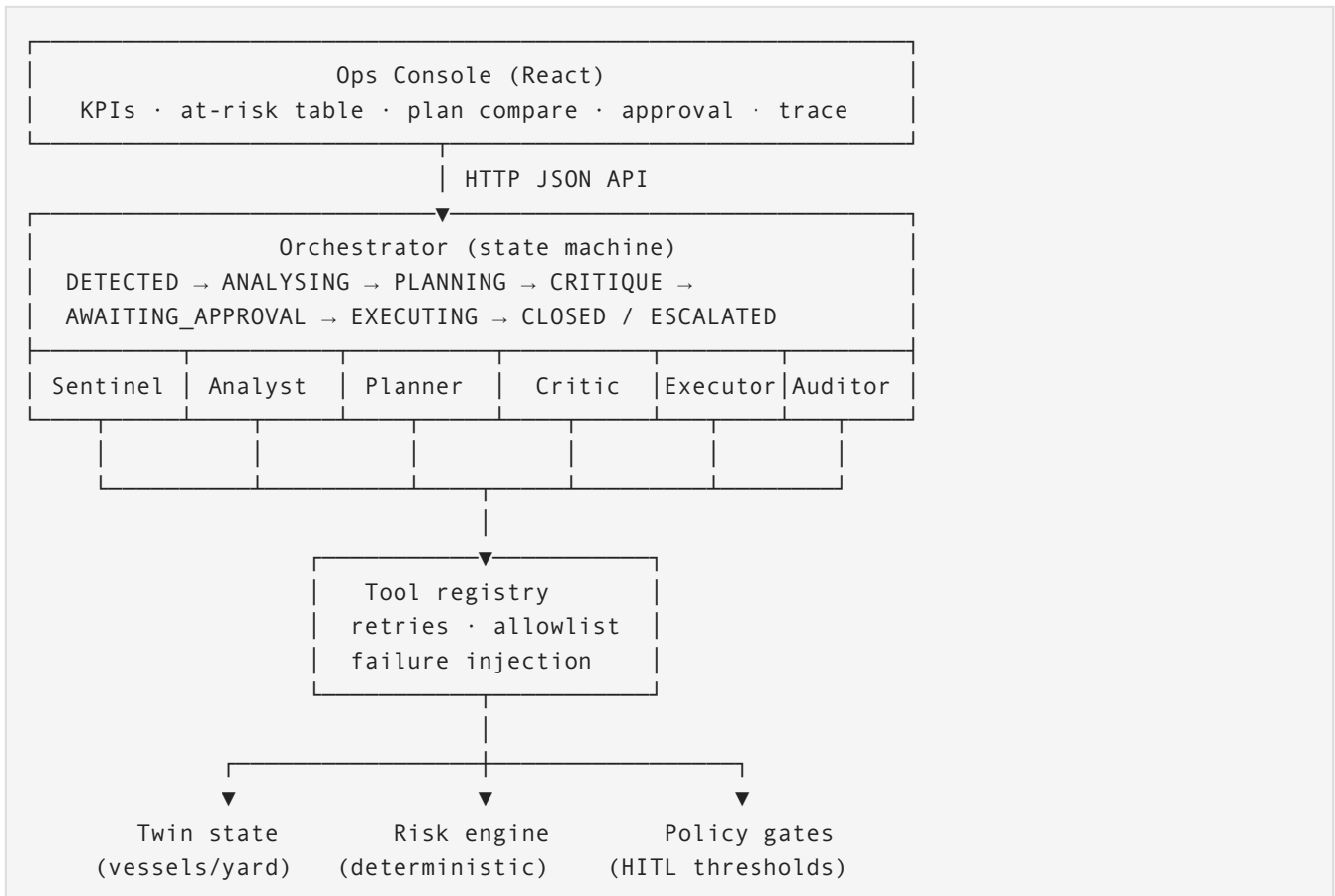
2. What RELAY does

RELAY is a human-in-the-loop agent workflow for connection recovery. In plain terms:

- Something triggers an incident (ETA change, crane status, bad data).
- Agents pull terminal state and score which transshipment links are inside the miss window.
- The system generates a few recovery plans and simulates them against the twin.
- A critic pass can reject plans that look good on paper but fail operationally.
- High-impact actions — restow, cutoff negotiation, holds — stop for planner approval.
- Everything is written to an execution trace: decisions, tool calls, errors, approvals, outcomes.

We did not aim for maximum autonomy. The kickoff brief is explicit that higher autonomy is not automatically better. Notifications and analysis can run on their own; moves that affect the yard or a sailing need a human sign-off.

3. System layout



Console — what the planner sees: at-risk boxes, plan comparison, approval buttons, live trace.

Orchestrator — a Python state machine we wrote ourselves. It moves an incident through defined phases and calls the right agent at each step. We kept this explicit rather than wrapping a generic agent framework, because the brief rewards workflow design the team actually thought through.

Domain engine — deterministic code for connection-buffer risk and plan simulation. The numbers in the demo are reproducible; agents coordinate tools, they do not invent the maths.

Tool layer — mock terminal APIs (schedule, yard, crane, work orders, notifications). Same shapes we would expect from real integration later.

4. How a run works (late feeder example)

Step	Who	What happens
1	Sentinel	Reads the trigger, updates feeder ETA in the twin, opens the incident
2	Analyst	Calls yard and connections tools; builds the at-risk list
3	Planner	Drafts PLAN-A/B/C, runs each through the simulator
4	Critic	Flags infeasible options (e.g. PLAN-X when a crane is down)
5	System	Runs low-risk notices; queues restow/cutoff actions behind approval
6	Planner (human)	Approves or rejects in the console
7	Executor + Auditor	Dispatches work orders with an approval token; re-scores; closes incident

If the planner rejects, the incident escalates to the shift superintendent instead of silently stopping.

5. Design choices (and why)

Transshipment-first. We wanted a PSA-specific problem, not a generic port chatbot. Connection buffer — time left before an outbound cutoff — is the metric planners actually care about.

Deterministic scoring + agent coordination. LLMs are good at narration; they are a poor place to put audited risk numbers. Our engine calculates buffer and simulates plans in Python. Agents decide what to call and when.

Separate critic agent. Planner and critic are split so dissent is real. The crane scenario shows PLAN-X getting rejected even though it scores well.

Approval tokens on dispatch. Work orders cannot go live without a token issued after human approval. You cannot bypass the gate by hitting the API directly.

No external dependencies for the demo. Backend is stdlib Python only — no pip install, no API keys. Judges can run it locally without setup pain.

Token estimates in the trace. We log approximate compute per step so efficiency is visible, not hand-waved.

6. Data and assumptions

PSA did not provide datasets, LLM keys, or credits for this round (organiser clarification on Telegram). We built a synthetic Tuas-style twin: vessels, ETAs, yard blocks, cranes, transshipment boxes, and a simple miss-cost model.

Source	Used for
Our twin	All runtime numbers in the demo
Public PSA/MPA stats	Problem framing only (~90% transshipment; ~90% off-schedule in 1H 2024; +8% rehandling)
External LLM	Not used at runtime

Assumptions we state openly:

- Connection buffer $\approx$ outbound cutoff minus feeder availability minus handling and a safety margin.
- Miss costs are illustrative (\$1,800 baseline with uplifts for premium/reefer/DG).
- Scenario 3 deliberately breaks the yard API so we can show retry $\rightarrow$ cache fallback.
- High-impact tools require an approval token.

More detail in presentation Appendix A.

7. Impact (demo twin — illustrative)

On our sample terminal, approved plans recover most or all at-risk connections in the first two scenarios:

Scenario	At-risk	Plan	Saved	Notes
Late feeder	7	PLAN-B	7	Cutoff extension + selective restow
Crane outage	7	PLAN-A	7	Critic kills infeasible PLAN-X
Uncertainty + API fail	12	PLAN-B	5	Partial recovery; residual risk left visible

These are not live PORTNET numbers. They show that the loop works end-to-end and that the critic and fallback paths change outcomes in sensible ways.

If shortlisted: we would run shadow mode with PSA mentors, tune weights with real planners, and swap mocks for PORTNET/CITOS-class APIs while keeping the same agent and policy contracts.

Where this fits publicly: MPA's Maritime Digital Twin is mainly situational awareness. RELAY sits on top as a recovery layer — scenario ranking, human gates, audit trail — when schedules break.

8. Security, safety, scaling

Access control (production path). Tools are allowlisted — no arbitrary shell or SQL. High-impact calls need an approval token. Console decisions are attributed (``decided_by``). Real deployment would add SSO and role-based access (duty officer / planner / superintendent).

Safety. DG and reefer flags tighten gates. Critic blocks capacity-infeasible plans. When AIS quality drops, uncertainty bands widen before scoring. Tool failures: retry, then fall back to last-known twin cache, then escalate if needed.

Scale. Risk scoring is $O(\text{containers})$ deterministic work — it does not scale linearly with LLM cost. Traces are append-only per incident, so you can audit without re-running agents. Multiple terminals could shard by cluster with a shared policy pack.

Errors. Retryable vs fatal tool errors are distinguished. Every failure shows up as a ``tool_error`` event in the trace. Rejection triggers superintendent notification with the full dossier.

9. Repo layout

```
relay/  
  backend/app/          # API, orchestrator, risk engine, policy, tools, scenarios  
  backend/tests/        # Risk, policy, end-to-end scenario tests  
  frontend/             # Ops console  
deliverables/           # Slides, this doc, demo scripts  
research/               # Brief notes, domain research, audit log
```

10. Run locally

```
# Terminal 1 – API
cd relay/backend && PYTHONPATH=. python3 -m app.main

# Terminal 2 – UI
cd relay/frontend && npm install && npm run dev
```

Open <http://localhost:5173>, run **Late feeder**, approve the plan, and follow the trace through to closed.